Authorization Deep Dive

Engines · Interceptor Patterns · XACML · OpenFGA · Zanzibar · gRPC · Envoy · Event-Driven

1.1 XACML Request/Response Model

```
john.smith@bank.com FINANCE true invoke-tool payment-approval-tool Permit HIGH
```

1.2 XACML vs Cedar — Structural Comparison

Dimension	XACML 3.0	AWS Cedar
Policy Format	Verbose XML — 50+ lines per rule	Concise DSL — 5-10 lines per policy
Combining Algorithms	14 combining algorithms (deny-overrides, permit-overrides, etc.)	Implicit: forbid overrides permit always
Obligations	Full obligation model in standard	Obligations via context and PEP conventions
Schema Validation	XML Schema (XSD)	Cedar schema — strongly typed
Performance	XML parsing overhead — typically 20-100ms	Sub-millisecond evaluation
Standards	OASIS standard — government/healthcare mandate	AWS proprietary (but open-source)
Formal Verification	No	Yes — Lean theorem prover
Ecosystem	IBM, Axiomatics, Forgerock implementations	AWS AVP only (growing ecosystem)
Use Today	Legacy enterprises, government mandates	New AWS-native projects

lationship-Based

Google Zanzibar (2019 paper) described the authorization system that powers Google Drive, YouTube, Calendar, and Maps — serving trillions of authorization checks per day. OpenFGA (Open Fine-Grained Authorization) is Auth0/Okta's open-source implementation of the Zanzibar model. Both solve a problem RBAC and ABAC cannot: authorization based on entity relationships.

2.1 The Relationship Model

```
// OpenFGA Authorization Model (Zanzibar-style) // Defines types and their allowed relations model schema 1.1 type user type agent relations define delegated_from: [user] type document relations define owner: [user] define editor: [user, agent] or owner define viewer: [user, agent] or editor define tenant_member: [user] type folder relations define owner: [user] define viewer: [user] or owner define parent: [folder] // Zanzibar Relationship Tuples (stored as facts): // user:john#owner@document:m-and-a-brief // user:jane#viewer@document:m-and-a-brief // user:john#member@tenant:bank-prod // agent:payments-agent#delegated_from@user:john //
```

Classification: CONFIDENTIAL — INTERNAL USE ONLY

Published: June 2026 · AWS Well-Architected Series

```
Authorization Check: // can agent:payments-agent view document:m-and-a-brief? // → agent is delegated_from user:john // → user:john is owner of document // → owner ⊇ viewer // → ALLOW
```

2.2 When ReBAC Beats RBAC and ABAC

ReBAC (Relationship-Based Access Control) is the superior model when authorization depends on the relationship between the principal and the resource, not just attributes:

Scenario	RBAC/ABAC Approach	ReBAC Approach
Can John edit his own documents?	Add 'document-editor' role globally — too broad	Tuple: john#editor@document:john-doc — precise
Can a manager see their team's reports?	Check department attribute match — brittle	Tuple: manager:jane#viewer@report-group:team-3
Can this agent access the user's calendar?	Add 'calendar-reader' capability — grants all calendars	Tuple: agent:X#delegated_reader@calendar:john-cal
Google Drive share with specific people	Create group for every share — explosion	Tuple per share: user:guest#viewer@doc:shared

■ NOTE

For Agentic AI: The most powerful ReBAC use case is agent-to-user-data relationships. Instead of 'agent has calendar-access capability' (too broad), use relationship tuples: 'agent:X is delegated_reader of user:john calendar'. This means the agent can ONLY access John's calendar for this session, not all calendars with that capability.

2.3 Hybrid: Cedar + OpenFGA

For enterprise Agentic AI, a pragmatic hybrid uses Cedar for business authorization policy and OpenFGA (or Cedar's own entity hierarchy) for relationship facts:

```
Architecture: OpenFGA (Relationship Store) Cedar (Policy Engine)
■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■ Stores FACTS: Evaluates POLICY: • user#owner@document • permit(principal, read, resource) • agent#delegated_from@user • when principal is owner of resource • team#member@project • and resource.classification <= user.clearance ↓ PIP Bridge ↓ OpenFGA check result flows into Cedar authorization context as a boolean attribute: context.isOwner = true/false context.isDelegatedFrom = true/false
```

3. PEP Interceptor Patterns — Complete Reference

The Policy Enforcement Point must be positioned at every boundary where authorization is required. This section provides implementation-level detail for each interceptor pattern.

3.1 API Gateway Lambda Authorizer (AWS Native)

```
# Lambda Authorizer — complete implementation pattern import json import boto3 import hashlib
import time from functools import lru_cache avp_client = boto3.client('verifiedpermissions')
POLICY_STORE_ID = 'ps-XXXXXXXXXXXXXXXXX' def handler(event, context): token =
event['authorizationToken'].replace('Bearer ', '') method_arn = event['methodArn'] # Extract
resource and action from ARN # arn:aws:execute-api:region:acct:api-id/stage/METHOD/resource
parts = method_arn.split('/') http_method = parts[2] resource_path = '/'.join(parts[3:]) try: #
Step 1: Validate JWT and get canonical claims canonical_claims = get_canonical_claims(token) #
Step 2: Build Cedar authorization request authz_request = build_cedar_request(
canonical_claims, http_method, resource_path, event ) # Step 3: Call Amazon Verified
Permissions response = avp_client.is_authorized(**authz_request) decision =
response['decision'] # 'ALLOW' or 'DENY' if decision == 'ALLOW': policy =
generate_allow_policy(method_arn, canonical_claims) log_decision('ALLOW', authz_request,
response) return policy else: log_decision('DENY', authz_request, response) raise
Exception('Unauthorized') # Returns 403 except Exception as e: log_decision('ERROR', {},
{'error': str(e)}) raise Exception('Unauthorized') # Default deny def
build_cedar_request(claims, method, path, event): return { 'policyStoreId': POLICY_STORE_ID,
'principal': { 'entityType': 'BankAI::User', 'entityId': claims['principal']['id'] }, 'action':
{ 'actionType': 'BankAI::Action', 'actionId': f'HTTP_{method}_{path.replace("/", "_")}' },
'resource': { 'entityType': 'BankAI::Resource', 'entityId': path }, 'context': { 'contextMap':
{ 'businessHours': {'boolean': is_business_hours()}, 'riskScore': {'long':
claims['context']['risk_score']}, 'mfaVerified': {'boolean':
claims['context']['mfa_verified']}, 'tenantId': {'string':
claims['organization']['tenant_id']}, 'geography': {'string':
claims['organization']['geography']}, } }, 'entities': { 'entityList':
build_entity_list(claims) } } def generate_allow_policy(method_arn, claims): # API Gateway IAM
policy — cached for 300s by API GW return { 'principalId': claims['principal']['id'],
'policyDocument': { 'Version': '2012-10-17', 'Statement': [{ 'Action': 'execute-api:Invoke',
'Effect': 'Allow', 'Resource': method_arn }] }, # Pass canonical claims downstream as context
'context': { 'tenantId': claims['organization']['tenant_id'], 'userId':
claims['principal']['id'], 'capabilities': json.dumps(claims['capabilities']) },
'usageIdentifierKey': claims['principal']['id'] }
```

3.2 Envoy / Istio External Authorization (Service Mesh)

For East-West (service-to-service) traffic in EKS/ECS, Envoy's external authorization filter calls an authorization service before forwarding requests. This is ideal for microservice-to-microservice agent calls:

```
# Envoy configuration: ext_authz filter # envoy-config.yaml http_filters: - name:
envoy.filters.http.ext_authz typed_config: "@type":
type.googleapis.com/envoy.extensions.filters.http.ext_authz.v3.ExtAuthz grpc_service:
envoy_grpc: cluster_name: authz_cluster timeout: 5s # Circuit breaker: timeout = deny
include_peer_certificate: true failure_mode_allow: false # CRITICAL: false = default deny
with_request_body: max_request_bytes: 8192 allow_partial_message: false # The authorization
service receives: # - All request headers (including JWT) # - Request path and method # - Source
principal (from mTLS certificate) # - Request body (if configured) # Authorization service
returns: # - 200 OK → forward request # - 403 Forbidden → block (with optional custom headers)
# OPA ext_authz implementation: # deploy as sidecar alongside Envoy # Policy bundle loaded from
S3 # Evaluation: <1ms (in-memory bundle)
```

3.3 FastAPI Middleware PEP

```
# FastAPI Middleware — PEP for Python agent services from fastapi import FastAPI, Request, HTTPException from fastapi.middleware.base import BaseHTTPMiddleware import httpx import json class AuthorizationMiddleware(BaseHTTPMiddleware): def __init__(self, app, avp_endpoint: str, policy_store_id: str): super().__init__(app) self.avp_endpoint = avp_endpoint self.policy_store_id = policy_store_id async def dispatch(self, request: Request, call_next): # Skip health check endpoints if request.url.path in ['/health', '/metrics']: return await call_next(request) # Extract canonical claims (set by Lambda Authorizer upstream) # Or validate JWT directly if no upstream authorizer claims = self._extract_claims(request) if not claims: raise HTTPException(status_code=401, detail="Missing identity") # Build Cedar authorization request authz_request = { 'policyStoreId': self.policy_store_id, 'principal': { 'entityType': 'BankAI::Agent', 'entityId': claims.get('agent_id', claims['user_id']) }, 'action': { 'actionType': 'BankAI::Action', 'actionId': self._map_action(request.method, request.url.path) }, 'resource': { 'entityType': 'BankAI::Resource', 'entityId': request.url.path }, 'context': { 'contextMap': { 'tenantId': {'string': claims['tenant_id']}, 'riskScore': {'long': claims.get('risk_score', 0)}, 'businessHours': {'boolean': self._is_business_hours()}, } } } # Call AVP (or cached decision) decision = await self._evaluate_policy(authz_request) if decision != 'ALLOW': # Log denial with full context await self._log_denial(request, claims, authz_request) raise HTTPException(status_code=403, detail="Authorization denied") # Attach claims to request state for downstream use request.state.claims = claims request.state.authorized = True # Execute handler response = await call_next(request) # Post-response: output classification check # (for sensitive data endpoints) return response def _extract_claims(self, request: Request) -> dict: # Canonical claims injected by upstream Lambda Authorizer # as custom headers or extracted from validated JWT claims_header = request.headers.get('X-Canonical-Claims') if claims_header: return json.loads(claims_header) return None
```

3.4 gRPC Interceptor PEP

```
# gRPC Unary Interceptor — for agent-to-agent gRPC calls import grpc import boto3 import json from functools import wraps avp = boto3.client('verifiedpermissions') class AuthorizationInterceptor(grpc.ServerInterceptor): def __init__(self, policy_store_id: str): self.policy_store_id = policy_store_id def intercept_service(self, continuation, handler_call_details): # Wrap the actual handler actual_handler = continuation(handler_call_details) if actual_handler is None: return None def auth_wrapper(request, context): # Extract JWT from gRPC metadata metadata = dict(context.invocation_metadata()) token = metadata.get('authorization', '').replace('bearer ', '') if not token: context.abort(grpc.StatusCode.UNAUTHENTICATED, 'Missing token') return None # Get canonical claims try: claims = get_canonical_claims(token) except Exception: context.abort(grpc.StatusCode.UNAUTHENTICATED, 'Invalid token') return None # Map gRPC method to Cedar action method_name = handler_call_details.method # e.g. '/bankai.AgentService/InvokeTool' -> 'InvokeTool' cedar_action = method_name.split('/')[-1] # Cedar authorization response = avp.is_authorized( policyStoreId=self.policy_store_id, principal={ 'entityType': 'BankAI::Agent', 'entityId': claims.get('agent_id', 'unknown') }, action={ 'actionType': 'BankAI::Action', 'actionId': cedar_action }, resource={ 'entityType': 'BankAI::Resource', 'entityId': metadata.get('resource-id', 'unknown') }, context={ 'contextMap': { 'tenantId': {'string': claims['organization']['tenant_id']}, 'riskScore': {'long': claims['context'].get('risk_score', 0)}, } } ) if response['decision'] != 'ALLOW': log_grpc_denial(method_name, claims, response) context.abort(grpc.StatusCode.PERMISSION_DENIED, 'Authorization denied') return None # Proceed to actual handler return actual_handler.unary_unary(request, context) return grpc.unary_unary_rpc_method_handler( auth_wrapper, request_deserializer=actual_handler.request_deserializer, response_serializer=actual_handler.response_serializer, )
```

4. Event-Driven Authorization Architecture

Multi-agent workflows often operate asynchronously via EventBridge or SQS. Authorization in event-driven systems requires a different pattern — the PEP cannot block in the traditional synchronous sense, but must validate events before they reach consumers.

4.1 Event Authorization Pipeline

```
Event Producer (Agent) ■ ■ Publishes event to EventBridge ■ Event includes: principal claims, action, resource, context ▼ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ EVENTBRIDGE AUTHORIZATION BUS ■ ■ ■ ■ EventBridge Rule → Authorization Lambda ■ ■ (Pre-consumer authorization check) ■ ■ ■ ■ Authorization Lambda: ■ ■ 1. Extract principal claims from event ■ ■ 2. Call Cedar AVP IsAuthorized ■ ■ 3. IF DENY → route to Dead Letter Queue (DLQ) ■ ■ + publish AUTHORIZATION_DENIED event■ ■ 4. IF ALLOW → forward to target consumer ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ (authorized events only) ▼ Consumer (Specialist Agent / Service) ■ ■ Re-validates on receipt (defense in depth) ■ A compromised EventBridge rule cannot bypass ■ consumer-side authorization ▼ Business Logic Execution
```

4.2 Event Authorization Lambda

```
# EventBridge Authorization Lambda import boto3 import json import logging avp = boto3.client('verifiedpermissions') eb = boto3.client('events') sqs = boto3.client('sqs') POLICY_STORE_ID = 'ps-XXXXXXXXXXXXXXXXXX' DLQ_URL = 'https://sqs.region.amazonaws.com/acct/authz-dlq' AUDIT_BUS = 'enterprise-audit-bus' def handler(event, context): # EventBridge delivers events in batches results = [] for record in event.get('Records', [event]): result = authorize_event(record) results.append(result) return results def authorize_event(event_detail): # Events MUST carry authorization context authz_ctx = event_detail.get('authorizationContext') if not authz_ctx: deny_event(event_detail, reason='MISSING_AUTH_CONTEXT') return {'status': 'DENIED', 'reason': 'MISSING_AUTH_CONTEXT'} principal = authz_ctx.get('principal') action = event_detail.get('detail-type', 'UNKNOWN_ACTION') resource = event_detail.get('resources', ['UNKNOWN'])[0] # Cedar evaluation response = avp.is_authorized( policyStoreId=POLICY_STORE_ID, principal={ 'entityType': 'BankAI::Agent', 'entityId': principal.get('agentId', 'unknown') }, action={ 'actionType': 'BankAI::Action', 'actionId': map_event_to_cedar_action(action) }, resource={ 'entityType': 'BankAI::Resource', 'entityId': resource }, context={'contextMap': build_context(authz_ctx)} ) if response['decision'] == 'ALLOW': # Log and allow event to proceed publish_audit_event(event_detail, 'ALLOWED', response) return {'status': 'ALLOWED'} else: deny_event(event_detail, response) return {'status': 'DENIED', 'reason': response.get('errors', [])} def deny_event(event_detail, response_or_reason): # Route denied events to DLQ for investigation sqs.send_message( QueueUrl=DLQ_URL, MessageBody=json.dumps({ 'event': event_detail, 'denial_reason': str(response_or_reason), 'timestamp': get_iso_timestamp() }) ) # Publish AUTHORIZATION_DENIED event for SIEM eb.put_events(Entries=[{ 'Source': 'bank.authorization', 'DetailType': 'AUTHORIZATION_DENIED', 'Detail': json.dumps({ 'event': event_detail, 'reason': str(response_or_reason) }), 'EventBusName': AUDIT_BUS }])
```

5. Complete STRIDE Threat Model for Enterprise Agentic AI

The following is a comprehensive STRIDE threat model for the entire authorization stack. Each threat includes its attack scenario, affected components, likelihood, impact, and specific mitigations implemented in this architecture.

5.1 Spoofing Threats

Threat	Attack Scenario	Affected Components	Mitigation
S1: Agent Identity Spoofing	Malicious code claims to be a trusted agent (e.g., 'payments-agent') by forging its entity ID in a Cedar request	Cedar PDP, MCP Gateway	Agent identity MUST be derived from mTLS certificate or signed IAM role credential — never from a self-reported header
S2: User Impersonation by Agent	Agent uses a stored user token to act beyond its delegation scope	Token Exchange Service	RFC 8693 delegation tokens are scope-constrained and carry act claim. Cedar verifies act.sub matches delegated scope.
S3: JWT Replay Attack	Attacker replays a captured user JWT to gain unauthorized access	Lambda Authorizer	JWT jti (JWT ID) tracked in ElastiCache for token lifetime. Duplicate jti = deny + alert.
S4: Tenant Claim Forgery	Attacker modifies JWT tenant_id claim to access another tenant's data	Claims Normalization	Tenant ID is cryptographically bound in the JWT signature. Only the IdP can set it. Normalization re-validates against IdP.
S5: JWKS Poisoning	Attacker substitutes a malicious JWKS endpoint to validate forged tokens	Lambda Authorizer	JWKS endpoint URL is hardcoded in authorizer config (not token-derived). Certificate pinning in production.

5.2 Tampering Threats

Threat	Attack Scenario	Affected Components	Mitigation
T1: Context Manipulation	Agent modifies risk_score or businessHours in the context object before Cedar evaluation	Claims Normalization, PDP	Context is constructed server-side from signed claims and server-clock. Client-supplied context values are rejected. Context hash is audited.
T2: Policy Store Tampering	Attacker modifies Cedar policies directly in AVP Console bypassing CI/CD	AVP Policy Store, PAP	AVP policy changes trigger CloudTrail alerts. Config rule detects drift. All changes require PRs. MFA for AVP console access.
T3: Audit Log Tampering	Attacker modifies or deletes CloudTrail logs to hide unauthorized actions	CloudTrail, S3	S3 Object Lock (WORM) on audit bucket. CloudTrail log file validation (SHA-256 chain). Log delivery to separate security account.
T4: Memory Injection	Agent writes malicious content to shared memory store that influences other agents	Memory Store, DynamoDB	Memory write authorization (Cedar policy). Content classification before write. Shared memory requires explicit project membership.

5.3 Repudiation Threats

Threat	Attack Scenario	Mitigation
R1: Tool Invocation Denial	Agent or user denies having invoked a destructive tool	CloudTrail records every AVP IsAuthorized call with principal, action, resource, decision, and policy matched. KMS-signed records are non-repudiable.
R2: Policy Change Denial	Administrator denies having changed a Cedar policy	AVP policy changes logged to CloudTrail with the IAM principal who made the change. Git commit history with signed commits provides additional non-repudiation.
R3: Data Access Denial	Agent denies having accessed a classified document via RAG	Post-retrieval Cedar evaluation logs every document access decision. Document chunk IDs and content hashes are in the audit record.

5.4 Information Disclosure Threats

Threat	Attack Scenario	Mitigation
I1: Cross-Tenant Data Leakage	Agent retrieves documents belonging to another tenant via RAG	Mandatory Cedar forbid on tenant mismatch. Pre-retrieval vector filter by tenant_id. Defense-in-depth: storage partition isolation.
I2: Over-Privileged LLM Context	Retrieved documents exceed the user's clearance level and are injected into LLM context	Post-retrieval Cedar per-chunk authorization removes over-classified chunks before context injection.
I3: Authorization Decision Leakage	Error messages reveal policy structure (e.g., 'You lack Finance_Approvers group membership')	All denial responses return generic 403. Detailed denial reasons logged to CloudTrail only. No policy internals in API responses.
I4: Memory Cross-User Leakage	Agent reads another user's episodic memory by manipulating session context	Memory records keyed by userId#tenantId in DynamoDB. Cedar forbid on memory.ownerId != principal.userId.
I5: PII in Agent Output	LLM generates response containing PII extracted from retrieved documents	Bedrock Guardrails + Amazon Macie scan agent outputs. Cedar output classification policy. DLP-failed outputs are blocked and flagged.

5.5 Denial of Service Threats

Threat	Attack Scenario	Mitigation
D1: PDP Exhaustion	Attacker floods AVP with IsAuthorized calls to exhaust capacity and cause deny-all	AVP is managed and auto-scales. Lambda Authorizer caches decisions (300s TTL). Rate limiting at WAF and API Gateway. Circuit breaker: if AVP unavailable → default deny (not open).
D2: Cache Poisoning DoS	Attacker triggers massive cache invalidation by causing repeated policy changes	Policy changes require CI/CD pipeline (multi-step, multi-approver). Emergency cache flush requires separate IAM permission. Rate limit on policy change events.
D3: Tool Invocation Storms	Runaway agent invokes tools in a tight loop, exhausting downstream API quotas	Per-agent, per-tool rate limiting enforced at MCP PEP Gateway. Step Functions workflow has max concurrency. Dead man's switch: agent execution time limit.

5.6 Elevation of Privilege Threats

Threat	Attack Scenario	Mitigation
E1: Confused Deputy Attack	Orchestrator agent accumulates permissions from multiple workflow steps, gaining more access than any single step should allow	Cedar evaluates the delegated scope at EVERY step independently. No permission accumulation between steps. Delegation tokens are scoped and time-limited.
E2: Prompt Injection Privilege Escalation	Adversarial content in retrieved documents instructs agent to invoke tools it is not authorized for	Bedrock Guardrails detect injection patterns. Cedar tool authorization is independent of LLM reasoning — a prompt cannot grant Cedar permissions.
E3: Agent-to-Agent Privilege Escalation	Sub-agent claims broader scope than the orchestrator delegated	Cedar verifies that sub-agent's delegated scope is $\subseteq$ orchestrator's scope. Token exchange service enforces scope intersection at delegation time.
E4: Expired Session Privilege Retention	Agent continues executing after the user's session expires, using cached authorization decisions	JWT expiry is enforced at normalization. Cached decisions have TTL $\leq$ JWT expiry. Re-authorization required for sessions > 60 minutes.

6. Authorization Performance Benchmarks

The following benchmarks are based on production measurements from AWS documentation, Styra DAS benchmarks, and published enterprise implementations. All figures assume warm cache unless stated otherwise.

6.1 Latency Benchmarks by Component

Component	P50	P95	P99	P99.9	Notes
AWS AVP IsAuthorized (managed)	2ms	5ms	8ms	15ms	Measured from Lambda in same region
OPA sidecar (policy bundle in-memory)	0.3ms	0.8ms	1.5ms	3ms	Rego evaluation, no network I/O
OPA central server (network)	1.5ms	3ms	5ms	12ms	Same-AZ, HTTP/2
Claims normalization (Redis hit)	0.5ms	1ms	2ms	4ms	ElastiCache same-AZ
Claims normalization (cold — LDAP)	15ms	25ms	40ms	80ms	Group GUID resolution from Entra
JWT signature validation (cached JWKS)	0.3ms	0.5ms	1ms	2ms	RSA-256 in Lambda memory
Lambda Authorizer total (cache hit)	1ms	2ms	4ms	8ms	API GW serves from IAM cache
Lambda Authorizer total (cache miss)	25ms	40ms	60ms	100ms	Cold path including AVP call
RAG pre-filter construction	0.2ms	0.5ms	1ms	2ms	Derived from cached claims
Cedar chunk post-auth (per chunk)	1ms	2ms	4ms	8ms	Batch mode for multiple chunks

6.2 Cache Hit Rate Targets

Cache Layer	Target Hit Rate	If Miss	Impact of Miss
API GW IAM Policy Cache (300s)	>70%	Lambda Authorizer executes	+40ms avg
Claims Normalization (Redis, 3600s TTL)	>95%	Full LDAP + mapping pipeline	+30ms avg
PIP Attribute Cache (Redis, 300s TTL)	>85%	DynamoDB attribute lookup	+5ms avg
OPA Bundle (in-memory, 30s refresh)	100%	N/A (async refresh)	No impact on request path
Cedar AVP (no client-side cache)	N/A (managed)	Always calls AVP API	AVP manages internally

■ NOTE

Performance Engineering Principle: The critical path for 95% of requests must be <5ms additional latency. This is achievable with warm caches. The 5% cold-path cost (25-60ms) is acceptable because it occurs only on first request in a new session. Never sacrifice security for performance — instead optimize the cache warm path.

6.3 Throughput & Scaling Characteristics

Component	Single Instance TPS	Horizontal Scaling	AWS Managed?
AVP IsAuthorized	~5,000 TPS per region	AWS manages — effectively unlimited	Yes — fully managed
Lambda Authorizer	1,000 concurrent (default)	Auto-scale — request AWS limit increase	Yes — serverless
OPA Sidecar (ECS)	~10,000 TPS per core	Scale with service instances (1:1)	No — self-managed
ElastiCache Redis (claims)	100,000+ ops/sec per node	Cluster mode — horizontal sharding	Yes — managed
Claims Normalization (ECS)	500 TPS per Fargate task (2vCPU)	Auto-scaling on CPU/request metrics	Yes — Fargate managed